

Python Automation

Module 2 — Traitement pandas automatisé

Data Analyst & Analytics Engineer Track

Datasets utilisés dans ce module :

`coffee_sales.csv` • `online_retail.csv` •
`comptage-voyageurs-trains-transilien.csv` • `TMDB_Data.csv`

0. Contexte et positionnement du module

Ce que tu as fait jusqu'ici (Mois 1 & 2)
Mois 1 — Fondations data : exploration et nettoyage manuels (NaN, types, doublons)
Mois 2 — E-sales : nettoyage manuel du Online Retail, valeurs bizarres dans la colonne produits
Module 1 Python Automation : organisation du système de fichiers (os, pathlib, shutil)
Résultat : tu comprends pourquoi on nettoie — il manque juste l'automatisation

Le Module 2 ne réapprend pas le nettoyage. Il encode ce que tu sais déjà faire à la main dans des scripts réutilisables. À la fin, tu auras des pipelines qui font en 3 secondes ce qui te prenait 20 minutes.

Progression dans le parcours

Module	Compétence	Ce que ça donne
Module 1 — Fichiers	Naviguer, organiser, renommer	Trouver et préparer les données
Module 2 — Pandas ◀	Lire, nettoyer, transformer	Pipeline de traitement automatisé
Module 3 — APIs	Récupérer des données externes	Données fraîches et enrichies
Module 4 — Pipelines	Orchestrer tout en un flux	Script de bout en bout

1. Chargement automatique de fichiers CSV

La première étape de tout travail data automatisé est de charger les données en mémoire. pandas le fait en une ligne, et c'est là que tout commence.

1.1 Import et chargement de base

```
import pandas as pd
from pathlib import Path

# Charger coffee_sales.csv
chemin = Path(r'C:/Users/toi/datasets/coffee_sales.csv')
df = pd.read_csv(chemin)
```

La variable `df` est un `DataFrame` — une table en mémoire avec des lignes indexées et des colonnes nommées. C'est sur elle que tout le traitement va s'appliquer.

1.2 Paramètres utiles de `read_csv`

Paramètre	Utilité
<code>sep=';'</code>	Si le CSV utilise le point-virgule comme séparateur
<code>encoding='utf-8'</code>	Pour les fichiers avec accents (transilien)
<code>parse_dates=['date']</code>	Convertir une colonne en format date automatiquement
<code>dtype={'id': str}</code>	Forcer le type d'une colonne au chargement
<code>nrows=1000</code>	Charger seulement les N premières lignes (test rapide)

```
# Exemple avec le fichier transilien (encodage et séparateur)
df_transilien = pd.read_csv(
    chemin_transilien,
    sep=';',
    encoding='utf-8',
    parse_dates=['date']
)
```

1.3 Chargement dynamique avec pathlib (Module 1 + Module 2)

En combinant pathlib (Module 1) et pandas, on peut charger automatiquement tous les CSV d'un dossier sans jamais écrire un chemin en dur.

```
# Charger tous les CSV du dossier datasets en un dictionnaire
from pathlib import Path
import pandas as pd
```

```
dossier = Path(r'C:/Users/toi/datasets')
dataframes = {}
```

```
for fichier in dossier.glob('*.csv'):
    nom = fichier.stem # nom sans extension
    dataframes[nom] = pd.read_csv(fichier, encoding='utf-8')
    print(f'Chargé : {nom} - {len(dataframes[nom])} lignes')
```

Résultat : dataframes['coffee_sales'], dataframes['TMDB_Data'], etc. Chaque nouveau CSV ajouté dans le dossier est chargé sans modifier le code.

2. Inspection automatisée des données

Avant de nettoyer quoi que ce soit, on inspecte. L'objectif est de générer automatiquement un rapport d'état sur chaque dataset — exactement ce que tu faisais à la main dans tes projets Mois 1 et Mois 2.

2.1 Les commandes d'inspection essentielles

Commande	Ce qu'elle donne
<code>df.shape</code>	Nombre de lignes et colonnes (ex : (1000, 12))
<code>df.dtypes</code>	Type de chaque colonne (int, float, object, datetime)
<code>df.info()</code>	Vue d'ensemble : types + valeurs non-nulles
<code>df.describe()</code>	Statistiques descriptives pour les colonnes numériques
<code>df.head(10)</code>	Les 10 premières lignes
<code>df.isnull().sum()</code>	Nombre de valeurs manquantes par colonne
<code>df.duplicated().sum()</code>	Nombre de lignes dupliquées
<code>df['col'].value_counts()</code>	Distribution des valeurs d'une colonne catégorielle

2.2 Fonction d'inspection automatique

Plutôt que de taper ces commandes une par une à chaque fois, on les encapsule dans une fonction réutilisable.

```
def inspecter(df, nom_dataset):  
    print(f'\n=== RAPPORT : {nom_dataset} ===')  
    print(f'Dimensions      : {df.shape[0]} lignes x {df.shape[1]}  
colonnes')  
    print(f'Doublons          : {df.duplicated().sum()}')  
    print(f'\nValeurs manquantes par colonne :')  
    nulls = df.isnull().sum()  
    print(nulls[nulls > 0].to_string())  
    print(f'\nTypes de colonnes :')  
    print(df.dtypes.to_string())  
    print('=' * 40)
```

```
# Utilisation sur coffee_sales  
inspecter(df_coffee, 'coffee_sales')
```

Cette fonction devient une brique de ton pipeline. Tu l'appelleras à chaque étape pour voir l'état des données avant et après nettoyage.

3. Nettoyage automatisé des données

C'est le coeur du module. On automatise les décisions de nettoyage que tu prenais à la main — NaN, doublons, types incorrects, valeurs aberrantes.

3.1 Gestion des valeurs manquantes (NaN)

Supprimer vs remplacer

La décision dépend du contexte métier, que tu maîtrises déjà grâce à ton Mois 1. On encode cette décision dans le script.

```
# Option 1 : supprimer les lignes avec NaN dans une colonne critique
df = df.dropna(subset=['product', 'quantity'])
```

```
# Option 2 : remplacer les NaN numériques par la médiane
df['unit_price'] = df['unit_price'].fillna(df['unit_price'].median())
```

```
# Option 3 : remplacer les NaN texte par une valeur par défaut
df['description'] = df['description'].fillna('Non renseigné')
```

Rapport de nettoyage NaN

```
def nettoyer_nan(df, colonnes_critiques):
    avant = df.isnull().sum().sum()
    df = df.dropna(subset=colonnes_critiques)
    apres = df.isnull().sum().sum()
    print(f'NaN supprimés : {avant - apres}')
    return df
```

3.2 Suppression des doublons

```
# Supprimer les doublons complets
df = df.drop_duplicates()
```

```
# Supprimer les doublons sur une clé spécifique
df = df.drop_duplicates(subset=['invoice_no', 'stock_code'])
```

```
# Garder la dernière occurrence (utile pour les mises à jour)
df = df.drop_duplicates(subset=['id'], keep='last')
```

3.3 Correction des types de colonnes

Un problème très courant : une colonne numérique chargée comme texte (object), ou une date chargée comme chaîne. C'est exactement ce type d'anomalie que tu as rencontré dans Online Retail.

```
# Convertir une colonne en numérique (erreurs → NaN)
df['unit_price'] = pd.to_numeric(df['unit_price'], errors='coerce')
```

```
# Convertir en date
df['invoice_date'] = pd.to_datetime(df['invoice_date'],
errors='coerce')
```

```
# Nettoyer et convertir une colonne avec des espaces ou symboles
df['price'] = df['price'].str.replace(',', '.').str.strip()
df['price'] = pd.to_numeric(df['price'], errors='coerce')
```

3.4 Détection des valeurs aberrantes

Tu as rencontré des valeurs bizarres dans la colonne produits du Online Retail. Voici comment les détecter et les traiter automatiquement.

```
# Détecter les quantités négatives (retours ou erreurs)
retours = df[df['quantity'] < 0]
print(f'Lignes avec quantité négative : {len(retours)}')
```

```
# Filtrer les valeurs aberrantes sur un seuil
df_propre = df[df['unit_price'] > 0]
```

```
# Détecter les codes produits au format inattendu
masque_invalides = ~df['stock_code'].str.match(r'^[A-Z0-9]+$',
na=False)
print(df[masque_invalides]['stock_code'].value_counts().head(10))
```


4. Transformations et enrichissement

Une fois les données propres, on les enrichit : nouvelles colonnes calculées, filtres, agrégations. C'est ici que pandas rejoint la logique SQL que tu connais déjà.

4.1 Créer de nouvelles colonnes

```
# Calculer le chiffre d'affaires par ligne
df['revenue'] = df['quantity'] * df['unit_price']
```

```
# Extraire l'année et le mois depuis une date
df['annee'] = df['invoice_date'].dt.year
df['mois'] = df['invoice_date'].dt.month
```

```
# Catégoriser une valeur numérique
df['segment_prix'] = pd.cut(
    df['unit_price'],
    bins=[0, 5, 20, 100, float('inf')],
    labels=['Bas', 'Moyen', 'Haut', 'Premium']
)
```

4.2 Filtrer les données

```
# Filtre simple : garder un pays
df_france = df[df['country'] == 'France']
```

```
# Filtre combiné (ET)
df_filtre = df[(df['quantity'] > 0) & (df['unit_price'] > 0)]
```

```
# Filtre sur une liste de valeurs
pays_cibles = ['France', 'Germany', 'Spain']
df_europe = df[df['country'].isin(pays_cibles)]
```

4.3 Agrégations (l'équivalent du GROUP BY SQL)

```
# CA total par pays
ca_par_pays = df.groupby('country')['revenue'].sum().reset_index()
ca_par_pays.columns = ['country', 'total_revenue']
ca_par_pays = ca_par_pays.sort_values('total_revenue',
ascending=False)
```

```
# Plusieurs agrégations en une fois
resume = df.groupby('country').agg(
    nb_commandes=('invoice_no', 'nunique'),
    ca_total=('revenue', 'sum'),
    panier_moyen=('revenue', 'mean')
).reset_index()
```

4.4 Équivalences pandas / SQL

Opération	SQL	pandas
Sélectionner	SELECT col1, col2	df[['col1', 'col2']]
Filtrer	WHERE col > 100	df[df['col'] > 100]
Trier	ORDER BY col DESC	df.sort_values('col', ascending=False)
Grouper	GROUP BY col	df.groupby('col').agg(...)
Joindre	JOIN table2 ON ...	pd.merge(df1, df2, on='id')
Dédoublonner	DISTINCT	df.drop_duplicates()

5. Export et sauvegarde des résultats

Un pipeline sans export ne sert à rien. On sauvegarde les données propres et on génère un log de ce qui a été fait.

5.1 Export CSV et Excel

```
# Export CSV propre (sans l'index pandas)
df_propre.to_csv('coffee_sales_clean.csv', index=False,
encoding='utf-8')

# Export Excel avec plusieurs onglets
with pd.ExcelWriter('rapport_module2.xlsx') as writer:
    df_propre.to_excel(writer, sheet_name='Données propres',
index=False)
    resume.to_excel(writer, sheet_name='Agrégations', index=False)
```

5.2 Génération automatique du nom de fichier

```
# Nom de fichier avec timestamp pour éviter les écrasements
from datetime import datetime
timestamp = datetime.now().strftime('%Y%m%d_%H%M')
nom_export = f'coffee_sales_clean_{timestamp}.csv'
df_propre.to_csv(nom_export, index=False)
```

5.3 Log du pipeline

```
# Garder une trace de toutes les transformations appliquées
log = []
log.append(f'Chargement : {len(df)} lignes')
log.append(f'Après suppression NaN : {len(df_sans_nan)} lignes')
log.append(f'Après suppression doublons : {len(df_propre)} lignes')

with open('pipeline_log.txt', 'w') as f:
    f.write('\n'.join(log))
```

6. Pipeline complet — mise en pratique

On assemble toutes les briques en un script unique, réutilisable sur n'importe lequel de tes datasets. C'est l'objectif final du module.

6.1 Structure du script pipeline

```
from pathlib import Path
import pandas as pd
from datetime import datetime

# — CONFIGURATION —
FICHIER_SOURCE = Path(r'C:/Users/toi/datasets/coffee_sales.csv')
DOSSIER_EXPORT = Path(r'C:/Users/toi/datasets/clean')
DOSSIER_EXPORT.mkdir(exist_ok=True)

# — ÉTAPE 1 : CHARGEMENT —
df = pd.read_csv(FICHIER_SOURCE, encoding='utf-8')
log = [f'Chargé : {len(df)} lignes, {df.shape[1]} colonnes']

# — ÉTAPE 2 : INSPECTION —
inspector(df, FICHIER_SOURCE.stem)

# — ÉTAPE 3 : NETTOYAGE —
df = df.drop_duplicates()
df = df.dropna(subset=['product', 'quantity'])
df['unit_price'] = pd.to_numeric(df['unit_price'], errors='coerce')
log.append(f'Après nettoyage : {len(df)} lignes')

# — ÉTAPE 4 : TRANSFORMATIONS —
df['revenue'] = df['quantity'] * df['unit_price']
df['mois'] = pd.to_datetime(df['date']).dt.month

# — ÉTAPE 5 : EXPORT —
ts = datetime.now().strftime('%Y%m%d_%H%M')
fichier_export = DOSSIER_EXPORT /
f'{FICHIER_SOURCE.stem}_clean_{ts}.csv'
df.to_csv(fichier_export, index=False)
log.append(f'Exporté : {fichier_export.name}')

# — LOG —
(DOSSIER_EXPORT / 'pipeline_log.txt').write_text('\n'.join(log))
```

```
print('Pipeline terminé.')
```

6.2 Ce que ce script devient au Mois 5

Ce pipeline manuel sera, au Mois 5, orchestré par Prefect : chaque étape deviendra un `task()` déclenché automatiquement selon un planning. Docker assurera que le script tourne dans le même environnement partout. Tu ne réécris pas — tu orchestres ce que tu as déjà construit.

Connexion Mois 5 — Prefect + Docker

Ton pipeline → une fonction Prefect `@flow` avec des `@task` pour chaque étape

Ton `FICHER_SOURCE` → un paramètre passé dynamiquement au déclenchement

Ton `log.txt` → remplacé par le tableau de bord Prefect (logs centralisés)

Docker → garantit que ton script tourne identiquement en local et en production

7. Exercices pratiques

Les exercices utilisent exclusivement des vrais datasets. Chaque exercice produit un fichier réel que tu peux committer dans ton GitHub.

Exercice 1 — Inspection automatique (coffee_sales.csv)

- Charger coffee_sales.csv avec pandas
- Appeler la fonction inspecter() et noter les observations
- Identifier les colonnes avec des NaN et les colonnes au mauvais type
- Exporter le résultat de df.describe() dans un fichier texte

Exercice 2 — Nettoyage (online_retail.csv)

- Charger le dataset Online Retail
- Identifier et supprimer les lignes avec une quantité négative
- Détecter les codes produits au format invalide (comme tu l'avais fait manuellement)
- Supprimer les doublons et les lignes sans CustomerID
- Exporter le dataset propre avec un timestamp dans le nom

Exercice 3 — Transformation et agrégation (TMDB_Data.csv)

- Charger TMDB_Data.csv
- Créer une colonne 'decade' à partir de l'année de sortie
- Calculer le score moyen par genre
- Filtrer les films avec un budget > 0 et un revenu > 0
- Exporter un CSV de synthèse avec les agrégations

Exercice 4 — Pipeline complet (transilien.csv)

- Adapter le script pipeline de la Section 6 pour le dataset transilien
- Gérer l'encodage et le séparateur spécifiques
- Ajouter une colonne 'heure_tranche' (matin / après-midi / soir)
- Générer le log complet et committer le tout dans ton repo GitHub

Récapitulatif — Ce que tu sais faire après ce module

Compétences acquises — Module 2
Charger n'importe quel CSV/Excel avec pandas, avec gestion de l'encodage et du séparateur
Générer un rapport d'inspection automatique (types, NaN, doublons, statistiques)
Nettoyer automatiquement : NaN, doublons, types incorrects, valeurs aberrantes
Créer des colonnes calculées et appliquer des transformations
Agréger des données (GROUP BY) et comprendre la logique SQL/pandas
Exporter des fichiers propres avec log et timestamp
Assembler le tout dans un pipeline reproductible et réutilisable

Ces scripts sont des briques directement réutilisables dans tes projets Mois 1, 2, et dans le pipeline Prefect du Mois 5. Tu n'écris pas du code jetable — tu construis ton stack Analytics Engineer.